Notes JavaScript

EIC95

Commentaires

Les commentaires sur une seule ligne commencent par **//** et les commentaires sur plusieurs lignes commencent par **/*** et se terminent par ***/**.

```
<!DOCTYPE html>
<html>
<body>
    <script>
        //comments
        /*
            comments
        */
    </script>
</body>
</html>
```

Affichage de données

JavaScript peut "afficher" des données de différentes manières :

- La méthode **document.getElementById("id")** est utilisée pour accéder à un élément HTML en JavaScript, et la propriété **innerHTML** définit son contenu HTML.

```html
<!DOCTYPE html>
<html>
<body>
    <h1>My First Web Page</h1>
    <p>My First Paragraph</p>
    <p id="demo"></p>
    <script>
        document.getElementById("demo").innerHTML = 5 + 6;
    </script>
</body>
</html>
```

- L'utilisation de **document.write()** est appropriée pour les tests, mais devrait être évitée dans les documents HTML chargés, car cela efface le HTML existant.

```html
<!DOCTYPE html>
<html>
<body>
    <h1>My First Web Page</h1>
    <p>My first paragraph.</p>
    <script>
        document.write(5 + 6);
    </script>
</body>
</html>
```

- **window.alert()** peut être utilisé pour afficher des données dans une boîte de pop-up, avec le mot clé window étant optionnel en JavaScript.

```html
<!DOCTYPE html>
<html>
<body>
    <h1>My First Web Page</h1>
    <p>My first paragraph.</p>
    <script>
        window.alert(5 + 6);
    </script>
</body>
</html>
```

- **console.log()** est utilisé pour les besoins de débogage, en affichant des données dans la console du navigateur.

```html
<!DOCTYPE html>
<html>
<body>
    <script>
        console.log(5 + 6);
    </script>
</body>
</html>
```

- JavaScript ne dispose pas d'un objet d'impression ni de méthode d'impression. La seule exception est que vous pouvez appeler la méthode **window.print()** dans le navigateur pour imprimer le contenu de la fenêtre actuelle.

```html
<!DOCTYPE html>
<html>
<body>
    <button onclick="window.print()">Print this page</button>
</body>
</html>
```

Variables

Il existe quatre façons de déclarer des variables : **var**, **let**, **const** et en ne déclarant pas (voir exemple). Il est considéré comme une bonne pratique de toujours déclarer les variables avant de les utiliser. Les variables peuvent stocker des nombres, du texte et d'autres types de données.

```
<!DOCTYPE html>
<html>
<body>
    <script>
        x = 5;
        y = 6;
        z = x + y;
    </script>
</body>
</html>
```

Les variables déclarées avec **var** ont une portée globale, ce qui signifie qu'elles sont accessibles depuis n'importe où dans le programme. Elles n'ont pas besoin d'être déclarées avant d'être utilisées et peuvent être redéclarées, ce qui peut entraîner des problèmes car cela changera la variable dans tout le programme. var ne doit être utilisé que dans le code écrit pour les navigateurs plus anciens.

```
<!DOCTYPE html>
<html>
<body>
    <script>
        var x = 5;
    </script>
</body>
</html>
```

Le mot-clé **let** fournit une portée de bloc, ce qui signifie que les variables déclarées avec let ne sont accessibles que dans le bloc dans lequel elles sont déclarées. Elles ne peuvent pas être redéclarées dans le même bloc et doivent être déclarées avant d'être utilisées.

```
<!DOCTYPE html>
<html>
<body>
    <script>
        let x = 5;
    </script>
</body>
</html>
```

const est utilisé lorsque vous savez que la valeur d'une variable ne devrait pas changer. Il a une portée de bloc, ne peut pas être redéclarées dans le même bloc, et ne peut pas être réassigné dans le même bloc.

```
<!DOCTYPE html>
<html>
<body>
    <script>
        const x = 5;
    </script>
</body>
</html>
```

Opérateurs

Arithmétiques

Opérateur	Description
+	Addition
-	Soustraction
*****	Multiplication
******	Exponentiation
/	Division
%	Modulo (reste de la division)
++	Incrémentation
--	Décrémentation

Afféctation

Opérateur	Exemple	Signification
=	x = y	x = y
+=	x += y	x = x + y
-=	x -= y	x = x - y
***=**	x *= y	x = x * y
/=	x /= y	x = x / y
%=	x %= y	x = x % y
****=**	x **= y	x = x ** y

Comparaison

Opérateur	Description en français
==	égal à
===	même valeur et même type
!=	différent de
!==	valeur différente ou type différent
>	supérieur à
<	inférieur à
>=	supérieur ou égal à
<=	inférieur ou égal à
?	opérateur ternaire

Logique

Opérateur	Description
&&	et logique
||	ou logique
!	non logique

Type

Opérateur	Description
typeof	Retourne le type d'une variable
instanceof	Retourne vrai si un objet est une instance d'un type d'objet

Types de données

Type de données	Description
Number	Représente les nombres, incluant les entiers et les flottants
String	Chaîne de caractères
Boolean	Valeur booléenne : vrai ou faux
Object	Collection de paires clé-valeur
Array	Liste ordonnée de valeurs
Function	Bloc de code réutilisable
Null	Valeur intentionnellement vide
Undefined	Valeur non définie
Symbol	Valeur unique et immuable
BigInt	Représente des entiers de taille arbitraire

Conversion de types

Il y a deux types principaux de conversions de type : implicite et explicite.

- **Conversion Implicite**: La conversion implicite (ou automatique) se produit lorsque JavaScript convertit automatiquement une valeur d'un type à un autre lorsque nécessaire. Cela peut souvent se produire dans des opérations mathématiques ou des comparaisons.

```
console.log('5' + 5);  // "55"
console.log('5' - 3);  // 2
console.log(true + 1); // 2
```

Conversion Explicite: La conversion explicite (ou casting) se produit lorsque vous convertissez manuellement une valeur d'un type à un autre en utilisant des méthodes ou des opérateurs.

En Nombre	**Number(value)**	Number("123") // 123
	parseInt(value, base)	parseInt("123", 10) // 123
	parseFloat(value)	parseFloat("123.45") // 123.45

En Chaîne	**String(value)**	String(123) // "123"
	value.toString()	123..toString() // "123"
En Booléen	**Boolean(value)**	Boolean("hello") // true
	!!value	!!1 // true

```
let c = '20';
let d = Number(c);
console.log(d + 10); // 30

let e = 15;
let f = String(e);
console.log(f + ' dollars'); // "15 dollars"
```

Chaines de caractères

Les chaînes peuvent être déclarées de différentes manières :

- Les guillemets simples (') et doubles (") sont interchangeables.
- Les backticks (`) permettent d'utiliser des chaînes de modèle (template literals) et insérer des variables et des expressions directement dans une chaîne.

La concaténation peut se faire avec l'opérateur + ou les template literals.

```
let str1 = "John";
let str2 = 'Doe';
let str3 = `Hello ${str1} ${str2}`;
```

Pour inclure des guillemets ou des caractères spéciaux dans une chaîne, utilisez le caractère d'échappement (****).

\'	Apostrophe ou guillemet simple échappé
\"	Guillemets doubles échappés
****	Barre oblique inverse échappée
\n	Nouvelle ligne
\r	Retour chariot
\t	Tabulation horizontale

```
let quote = 'He said, "It\'s a beautiful day!"';
console.log(quote); //He said, "It's a beautiful day!"
```

Méthode	Description
.length	Retourne la longueur de la chaîne
.charAt(index)	Retourne le caractère à l'index spécifié
.includes(substring)	Vérifie si la chaîne contient la sous-chaîne
.indexOf(substring)	Retourne l'index de la première occurrence de la sous-chaîne
.slice(start, end)	Extrait une partie de la chaîne de start à end (non inclus)

.substring(start, end)	Extrait les caractères entre start et end (non inclus)
.toUpperCase()	Convertit la chaîne en majuscules
.toLowerCase()	Convertit la chaîne en minuscules
.trim()	Supprime les espaces blancs au début et à la fin
.replace(oldValue, newValue)	Remplace la première occurrence de oldValue par newValue
.split(separator)	Divise la chaîne en un tableau de sous-chaînes

Mathématiques

JavaScript fournit 8 constantes mathématiques accessibles en tant que propriétés mathématiques :

- **Math.E** : retourne le nombre d'Euler
- **Math.PI** : retourne PI
- **Math.SQRT2** : retourne la racine carrée de 2
- **Math.SQRT1_2** : retourne la racine carrée de 1/2
- **Math.LN2** : retourne le logarithme naturel de 2
- **Math.LN10** : retourne le logarithme naturel de 10
- **Math.LOG2E** : retourne le logarithme en base 2 de E
- **Math.LOG10E** : retourne le logarithme en base 10 de E

Méthode	Description
Math.abs(x)	Retourne la valeur absolue de x
Math.ceil(x)	Retourne le plus petit entier supérieur ou égal à x
Math.floor(x)	Retourne le plus grand entier inférieur ou égal à x
Math.round(x)	Retourne l'entier le plus proche de x

Math.max(a, b, ...)	Retourne le plus grand des arguments
Math.min(a, b, ...)	Retourne le plus petit des arguments
Math.random()	Retourne un nombre pseudo-aléatoire entre 0 (inclus) et 1 (exclus)
Math.pow(base, exponent)	Retourne base élevé à la puissance exponent
Math.sqrt(x)	Retourne la racine carrée de x

```
let PI = Math.PI;
let a = -5;
console.log(Math.abs(a));
```

Conditions

if

```
if (hour < 18) {
    greeting = "Good day";
}
```

if...else

```
if (hour < 18) {
    greeting = "Good day";
} else {
    greeting = "Good evening";
}
```

else if

```
if (time < 10) {
    greeting = "Good morning";
} else if (time < 20) {
    greeting = "Good day";
} else {
    greeting = "Good evening";
}
```

switch

```
let x = 0;
let text;
switch (x) {
    case 0:
        text = "Off";
        break;
    case 1:
        text = "On";
        break;
    default:
        text = "No value found";
}
```

Boucles

for

```
for (let i = 0; i < 5; i++) {
    console.log("The number is " + i + "\n");
}
```

L'instruction JavaScript **for in** parcourt les propriétés d'un objet. Chaque itération renvoie une clé (x).

```
const person = {fname:"John", lname:"Doe", age:25};
for (let x in person) {
    console.log(person[x]);
}
```

L'instruction JavaScript **for of** parcourt les valeurs d'un objet itérable.

```
const tab = [1,2,3,4,5] ;
for (let x of tab) {
    console.log(x);
}
```

while

```
let i = 0;
while (i < 10) {
    console.log("The number is " + i);
    i++;
}
```

do...while

```
let i = 0;
do{
    console.log("The number is " + i);
    i++;
}while (i < 10);
```

prompt()

La fonction **prompt()** en JavaScript affiche une boîte de dialogue modale qui demande à l'utilisateur de saisir une entrée.

```
let name = prompt("What's your name?");
console.log("Hello, " + name);
```

Tableaux

Utilisez des crochets **[]** pour créer un tableau et accédez aux éléments via leur index (commence à 0). Utilisez **length** pour connaitre le nombre d'éléments dans le tableau.

```
const tab = [1,2,3,4,5]
console.log(tab.length);
```

Méthode	Description
.push(element)	Ajoute un ou plusieurs éléments à la fin du tableau.
.pop()	Supprime le dernier élément du tableau.
.shift()	Supprime le premier élément du tableau.
.unshift(element)	Ajoute un ou plusieurs éléments au début du tableau.

.forEach(callback)	Exécute une fonction pour chaque élément du tableau.
.includes(element)	Vérifie si le tableau contient l'élément spécifié.

```
const tab = [1,2,3,4,5]
tab.pop();
console.log(tab[4]);//undefined
```

Fonctions

Les fonctions JavaScript sont définies avec le mot-clé **function**. Vous pouvez utiliser une déclaration de fonction ou une expression de fonction qui peut être stockée dans une variable.

```
const x = 5;
const y = 6;
function addition(a,b){
    return a + b;
}
const add = function (a , b){
    return a + b;
}
addition(x,y);
add(x,y);
```

Les fonction fléchée sont une syntaxe concise pour écrire des fonctions, surtout utile pour des fonctions simples.

```
const x = 5;
const y = 6;
const add = (a , b) => a + b;
add(x,y);
```

Vous pouvez définir des valeurs par défaut pour les paramètres.

```
function addition(a = 10 , b = 20){
    return a + b;
}
addition();
```

Classes et Objets

Les objets sont des collections de paires clé-valeur. Vous pouvez utiliser la notation par points ou crochets pour accéder aux propriétés. Vous pouvez ajouter ou modifier des propriétés existantes. Les méthodes sont des fonctions définies à l'intérieur d'un objet. Le mot-clé **this** est utilisé pour faire référence à l'objet courant.

```
let person = {
    firstname : "John",
    lastname : "Doe",
    age : 25,
    get : function(){
        return `${this.firstname} ${this.lastname}`;
    }
}
```

Les classes JavaScript sont des modèles pour les objets JavaScript. Utilisez le mot-clé **class** pour définir une classe. Utilisez le mot-clé **new** pour créer des instances d'une classe. Accédez aux méthodes et propriétés des instances de classe avec l'opérateur point (**.**).

```
class Person {
    constructor(firstName, lastName, age) {
        this.firstName = firstName;
        this.lastName = lastName;
        this.age = age;
    }
    greet() {
        console.log("Hello, " + this.firstName);
    }
}
let random = new Person("John" , "Doe" , 25);
```

Les classes peuvent hériter d'autres classes en utilisant le mot-clé **extends**.

```
class Person {
    constructor(firstName, lastName, age) {
        this.firstName = firstName;
        this.lastName = lastName;
        this.age = age;
    }
    greet() {
        console.log("Hello, " + this.firstName);
    }
}
class Certified_Person extends Person{
    constructor(firstName, lastName, age, id){
        super(firstName, lastName, age);
        this.id = id;
    }
}
let certified = new Certified_Person("Jane" , "Doe" , 25 , 123456789);
```

Vous pouvez définir des méthodes qui ne sont pas liées à une instance spécifique, mais à la classe elle-même, avec le mot-clé **static**.

```
class MyClass {
    static greet(a) {
        console.log("Hello, " + a);
    }
}
MyClass.greet("John");
```

Déstructuration

La destructuration d’itérables permet d'extraire des éléments d'un tableau et de les assigner à des variables. Les itérables sont des objets qui peuvent être parcourus à l'aide d'un itérateur. Vous pouvez renommer les variables lors de la destructuration , définir des valeurs par défaut pour les propriétés qui pourraient être undefined et ignorer certains éléments en les omettant dans la liste de variables.

```
const person = {
    name : "John",
    age : 25,
    height : 1.50
}
let {name:fullname,height,weight = 65} = person;
console.log(fullname);
```

Exceptions

Pour gérer des erreurs, JavaScript utilise les blocs **try...catch**. Le code susceptible de provoquer une erreur est placé dans le bloc try, et si une exception se produit, le contrôle est transféré au bloc **catch** où vous pouvez définir la manière de gérer l'erreur. Lorsque vous utilisez **throw**, vous interrompez l'exécution normale du code et déclenchez la gestion des erreurs dans un bloc catch correspondant. Il est également possible d'utiliser **finally** pour exécuter du code après le bloc try...catch, que l'exception ait été levée ou non.

```
function checkAge(age) {
    if (age < 18) {
    throw new Error("L'âge doit être supérieur ou égal à 18.");
    }
    return "Accès autorisé";
}

try {
    let result = checkAge(15);
    console.log(result);
} catch (error) {
    console.log("Une erreur est survenue:", error.message);
} finally {
    console.log("Vérification de l'âge terminée.");
}
```

Dates

Pour obtenir la date et l'heure actuelles, vous pouvez créer un nouvel objet **Date** sans paramètres. Vous pouvez créer une date spécifique en passant les paramètres année, mois, jour, heure, minute, seconde et milliseconde. Notez que le mois commence à 0 (0 pour janvier, 1 pour février, etc.).

```
const first_date = new Date();
const second_date = new Date(28,0,2006);
```

Vous pouvez aussi modifier les composants de la date avec les méthodes **set**.

```
let now = new Date();
now.setFullYear(2025);
now.setMonth(0);
now.setDate(15);
now.setHours(10);
now.setMinutes(30);
console.log("new date :", now);
```

Vous pouvez extraire des parties spécifiques de la date, comme l'année, le mois, le jour, etc.

```
let now = new Date();
console.log("Year:", now.getFullYear());
console.log("Month:", now.getMonth() + 1);
console.log("Day:", now.getDate());
console.log("Hours:", now.getHours());
console.log("Minutes:", now.getMinutes());
console.log("Seconds:", now.getSeconds());
```

Timers

En JavaScript, les timers permettent de programmer l'exécution de code après un certain délai ou de manière répétée à intervalles réguliers. Les deux principales méthodes pour travailler avec les timers sont setTimeout et setInterval :

- **setTimeout** exécute une fonction une seule fois après un délai spécifié.
- **setInterval** exécute une fonction de manière répétée à intervalles réguliers spécifiés.

Utilisez **clearTimeout** et **clearInterval** pour annuler les timers.

```
let timeoutId = setTimeout(() => {
console.log("This message appears after 2 seconds");
}, 2000);
clearTimeout(timeoutId);

let intervalId = setInterval(() => {
console.log("This message appears every 3 seconds");
}, 3000);
setTimeout(() => {
clearInterval(intervalId);
onsole.log("setInterval has been cleared");
}, 10000);
```

Modules

Les modules en JavaScript permettent de structurer et d'organiser le code en séparant les fonctionnalités en unités indépendantes. Les modules peuvent être importés et exportés entre différents fichiers pour améliorer la modularité et la réutilisabilité du code.

Exportation Nomée : Permet d'exporter plusieurs éléments depuis un fichier.

```
// file.js
export const myVar = 42;
export function myFunction() {
    return "Hello!";
}
```

Exportation par Défault : Permet d'exporter un seul élément par fichier.

```
// file.js
export default function() {
    return "Default export";
}
```

Importation Nomée : Permet d'importer des éléments spécifiques depuis un module.

```
<script type = "module">
    // main.js
    import { myVar, myFunction } from './file.js';
    console.log(myVar);          // 42
    console.log(myFunction());   // "Hello!"
</script>
```

Importation par Défault : Permet d'importer l'élément exporté par défaut d'un module.

```
<script type = "module">
    // main.js
    import myDefaultFunction from './file.js';
    console.log(myDefaultFunction()); // "Default export"
</script>
```

Renommer des Imports :

```
<script type = "module">
    // main.js
    import { myFunction as newFunctionName } from './file.js';
    console.log(newFunctionName()); // "Hello!"
</script>
```

Expressions régulières

Les expressions régulières (ou regex) sont des outils puissants pour la manipulation et la recherche de texte. Elles permettent de définir des motifs de recherche et de validation dans des chaînes de caractères en utilisant une syntaxe spéciale et de méthodes. Utilisez des caractères comme **.** pour n'importe quel caractère, **^** pour le début de chaîne, et **$** pour la fin.

Méthode	Description
test()	Vérifie si une chaîne correspond au motif.
exec()	Exécute une recherche pour le motif et retourne les résultats.
match()	Trouve les correspondances dans une chaîne.
replace()	Remplace les correspondances par une nouvelle chaîne.
split()	Divise une chaîne en un tableau de sous-chaînes basées sur le motif.

```
<script>
    const regex = /abc/;
    console.log(regex.test("abcdef")); // true
    const str = "abcdef";
    console.log(str.replace(/abc/, "XYZ")); // "XYZdef"
</script>
```

Set

En JavaScript, un Set est une collection d'éléments uniques qui permet de stocker des valeurs sans duplication. Contrairement aux tableaux, les ensembles (Sets) ne permettent pas d'avoir des éléments en double et offrent des opérations efficaces pour vérifier la présence d'un élément. Vous pouvez créer un Set en utilisant le constructeur **Set()**, qui peut accepter un tableau ou un autre objet iterable comme argument.

```
const mySet = new Set();
const anotherSet = new Set([1, 2, 3, 4, 4]);
```

add(value)	Ajoute une valeur au Set. Si la valeur existe déjà, elle n'est pas ajoutée.
delete(value)	Supprime une valeur du Set. Renvoie true si la valeur a été supprimée, sinon false.
has(value)	Vérifie si une valeur est présente dans le Set. Renvoie true ou false.
clear()	Supprime toutes les valeurs du Set.
size	Propriété qui retourne le nombre d'éléments dans le Set.

Vous pouvez itérer sur les éléments d'un Set en utilisant les méthodes **forEach()** qui exécute une fonction pour chaque élément du Set ou en utilisant des boucles **for...of**.

```
mySet.forEach(value => {
    console.log(value);
});
for (const value of mySet) {
    console.log(value);
}
```

Map

En JavaScript, un Map est une collection d'éléments où chaque élément est une paire clé-valeur. Les clés et les valeurs peuvent être de n'importe quel type, et contrairement aux objets, les clés d'un Map sont ordonnées et peuvent être de n'importe quel type, y compris des objets. Vous pouvez créer un Map en utilisant le constructeur **Map()**, qui peut accepter un tableau de paires clé-valeur. On peut executer une fonction pour chaque valeur avec **forEach()**.

set(key, value)	Ajoute ou met à jour une paire clé-valeur dans le Map.
get(key)	Retourne la valeur associée à la clé spécifiée.
has(key)	Vérifie si une clé est présente dans le Map. Renvoie true ou false.
delete(key)	Supprime la paire clé-valeur associée à la clé spécifiée. Renvoie true si la clé a été supprimée, sinon false.
clear()	Supprime toutes les paires clé-valeur du Map.
size	Propriété qui retourne le nombre d'éléments dans le Map.

```
const myMap = new Map();
const anotherMap = new Map([['key1', 'value1'], ['key2', 'value2']]);
myMap.set('name', 'John');
myMap.set('age', 30);
console.log(myMap.get('name')); // 'John'
```

Promesses

Les promesses (ou Promises en anglais) en JavaScript sont un mécanisme pour gérer les opérations asynchrones. Elles permettent de traiter des valeurs qui ne sont pas encore disponibles mais le seront probablement dans le futur. Une promesse peut être dans l'un des trois états suivants :

- **Pending** (En attente) : L'état initial. La promesse est en cours de traitement et n'est ni résolue ni rejetée.
- **Fulfilled** (Réussie) : La promesse a été résolue avec succès. Une valeur est maintenant disponible.
- **Rejected** (Rejetée) : La promesse a échoué. Une erreur est disponible.

Une promesse est créée en utilisant le constructeur Promise, qui accepte une fonction de rappel (callback) avec deux arguments : resolve et reject.

```
const myPromise = new Promise((resolve, reject) => {
    const success = true;

    if (success) {
        resolve('Operation was successful');
    } else {
        reject('Operation failed');
    }
});
```

Les promesses simplifient la gestion des opérations en permettant d'enchaîner les traitements avec **then()**, de capturer les erreurs avec **catch()**, et d'exécuter des actions finales avec **finally()**. Par exemple, avec une promesse de récupération de données, vous pouvez utiliser then() pour traiter les données lorsque la promesse est remplie, catch() pour gérer les erreurs éventuelles, et finally() pour exécuter du code après la résolution ou le rejet de la promesse.

```
const fetchData = new Promise((resolve, reject) => {
    setTimeout(() => {
      const success = true;
      if (success) {
        resolve('Data received');
      } else {
        reject('Failed to fetch data');
      }
    }, 1000);
  });

  fetchData
    .then(result => console.log(result)) // 'Data received'
    .catch(error => console.error(error)) // 'Failed to fetch data'
    .finally(() => console.log('Promise has been settled'));
```

DOM

Le Document Object Model (DOM) est une interface de programmation qui permet aux scripts d'accéder et de manipuler le contenu, la structure et le style des documents HTML et XML. Le DOM représente le document sous la forme d'une arborescence hiérarchique où chaque nœud est un objet représentant une partie du document (éléments, attributs, texte, etc.). Cette structure permet aux développeurs de naviguer dans le document et de modifier son contenu dynamiquement en utilisant des langages de programmation comme JavaScript.

Pour interagir avec les éléments du DOM, il est essentiel de pouvoir les sélectionner. Plusieurs méthodes sont disponibles pour cela : **getElementById** sélectionne un élément par son ID, **getElementsByClassName** sélectionne tous les éléments avec une certaine classe, et **getElementsByTagName** sélectionne les éléments par leur nom de balise. Les méthodes **querySelector** et **querySelectorAll** permettent d'utiliser des sélecteurs CSS pour trouver des éléments, offrant une grande flexibilité.

```
<script>
    const byId = document.getElementById('example');
    const byClass = document.getElementsByClassName('example');
    const byTag = document.getElementsByTagName('p');
    const firstBySelector = document.querySelector('.example');
    const allBySelector = document.querySelectorAll('p.example');
</script>
```

Une fois les éléments sélectionnés, vous pouvez modifier leur contenu et leurs attributs. La propriété **innerHTML** permet de changer le contenu HTML intérieur d'un élément, tandis que **textContent** modifie le texte à l'intérieur. Les attributs peuvent être manipulés avec **setAttribute** et **getAttribute**. Pour modifier le style, vous pouvez utiliser **style.propertyName** pour définir des propriétés CSS directement sur les éléments.

```
// modify contents
const byInnerHTML = document.getElementById('example').innerHTML = "Nouveau contenu";
const byTextContent = document.getElementById('example').textContent = "Nouveau texte";

// modify attributs
const bySetAttribute = document.getElementById('example').setAttribute('data-info', 'nouvelle valeur');
const byGetAttribute = document.getElementById('example').getAttribute('data-info');
console.log('Attribut data-info:', byGetAttribute);

// modify styles
const byStyleColor = document.getElementById('example').style.color = 'blue';
const byStyleFontSize = document.getElementById('example').style.fontSize = '18px';

// add or remove classes
const byAddClass = document.getElementById('example').classList.add('modified');
const byRemoveClass = document.getElementById('example').classList.remove('example');
const byContainsClass = document.getElementById('example').classList.contains('modified');
```

Les événements sont des actions que l'utilisateur ou le navigateur peut déclencher, comme des clics de souris ou des entrées de texte. Vous pouvez ajouter des écouteurs d'événements aux éléments avec **addEventListener**, qui exécute une fonction spécifique lorsque l'événement se produit. Les événements courants incluent **click** pour les clics de souris et **input** pour la saisie de texte, permettant de rendre les pages web interactives et réactives aux actions de l'utilisateur.

```html
<button onclick="random_function()">click</button>
<script>
    // Click Event
    document.getElementById('exampleButton').addEventListener('click', function() { /* code */ });
    // Input Event
    document.getElementById('exampleInput').addEventListener('input', function(event) { /* code */ });
    // Mouseover Event
    document.getElementById('exampleDiv').addEventListener('mouseover', function() { /* code */ });
    // Mouseout Event
    document.getElementById('exampleDiv').addEventListener('mouseout', function() { /* code */ });
    // Change Event
    document.getElementById('exampleInput').addEventListener('change', function() { /* code */ });
    // Keypress Event
    document.getElementById('exampleInput').addEventListener('keypress', function(event) { /* code */ });
    // Submit Event
    document.querySelector('form')?.addEventListener('submit', function(event) { /* code */ });
    // Focus Event
    document.getElementById('exampleInput').addEventListener('focus', function() { /* code */ });
</script>
```

Le DOM permet aussi de créer et de supprimer des éléments dynamiquement. La méthode **createElement** crée un nouvel élément HTML, et **createTextNode** crée un nœud de texte. Vous pouvez ajouter ces nouveaux nœuds au document avec **appendChild** ou **insertBefore**. Pour supprimer des éléments, utilisez **removeChild**, ce qui permet de retirer un nœud enfant d'un élément, et **remove** dans les navigateurs les plus récents, modifiant ainsi la structure du document en temps réel.

```
document.createElement('tagName'); // Create new element
document.createTextNode('text'); // Create text node
parentElement.appendChild(childElement); // Append child
parentElement.insertBefore(newElement, referenceElement); // Insert before
parentElement.removeChild(childElement); // Remove child
parentElement.remove(referenceElement); // Remove element
```

Dans le DOM, tout est un nœud. La racine de cette structure est le nœud document. Les nœuds d'éléments correspondent aux balises HTML, les nœuds d'attributs contiennent les propriétés des éléments, et les nœuds de texte représentent le texte à l'intérieur des éléments. Cette organisation permet de manipuler chaque partie du document de manière précise, en ajoutant, supprimant ou modifiant les différents nœuds.

Naviguer dans le DOM signifie accéder aux différents nœuds par leur relation hiérarchique. Vous pouvez accéder au parent d'un nœud avec **parentNode**, et aux enfants avec **childNodes**, **firstChild**, et **lastChild**. Les nœuds frères, ou "siblings", peuvent être atteints avec **nextSibling** et **previousSibling**. Cette navigation permet de parcourir et de manipuler les éléments de la page de manière structurée et organisée.

```
// Parent Node
console.log(document.getElementById('firstChild').parentNode);
// Child Nodes
console.log(document.getElementById('container').childNodes);
// First Child
console.log(document.getElementById('container').firstChild);
console.log(document.getElementById('container').lastChild);
// Next Sibling
console.log(document.getElementById('firstChild').nextSibling);
// Previous Sibling
console.log(document.getElementById('secondChild').previousSibling);
```

JSON

JSON (JavaScript Object Notation) est un format léger et textuel d'échange de données. Il est conçu pour être facilement lisible et modifiable par les humains, et simple à analyser et générer par les machines. JSON est souvent utilisé pour transmettre des données entre un client web et un serveur, notamment dans les API web, en raison de sa simplicité et de sa compatibilité avec de nombreux langages de programmation.

La structure de JSON repose principalement sur deux éléments : les objets et les tableaux. Les objets sont délimités par des accolades **{}** et contiennent des paires clé-valeur. Chaque clé est une chaîne de caractères suivie d'un deux-points, puis de la valeur correspondante. Les tableaux, quant à eux, sont définis par des crochets **[]** et peuvent contenir une liste ordonnée de valeurs, séparées par des virgules. Ces éléments de base permettent de construire des structures de données hiérarchiques et complexes.

```
[ "value1", "value2", "value3" ]

{ "key1": "value1", "key2": "value2" }
```

JSON prend en charge plusieurs types de données. Les chaînes de caractères doivent être entre guillemets doubles, tandis que les nombres peuvent être entiers ou à virgule flottante. Les objets et tableaux peuvent contenir des valeurs imbriquées, permettant de créer des structures complexes. Les valeurs booléennes (true et false) et la valeur null sont également prises en charge. Ces types de données offrent la flexibilité nécessaire pour représenter une grande variété d'informations.

En JavaScript, JSON est couramment utilisé pour manipuler les données récupérées des serveurs ou des API. La méthode **JSON.parse()** est utilisée pour convertir une chaîne JSON en objet JavaScript, tandis que **JSON.stringify()** permet de convertir un objet JavaScript en chaîne JSON. En PHP, les fonctions **json_decode()** et **json_encode()** remplissent des rôles similaires, facilitant la manipulation des données JSON côté serveur.

```
//json file
{
    "person": {
        "firstName": "Alice",
        "lastName": "Smith",
        "age": 30
    }
}
```

```
// JSON Data
const jsonData = '{"person": {"firstName": "Alice", "lastName": "Smith", "age": 30}}';

// Parse JSON
const data = JSON.parse(jsonData);

// Access Data
console.log(data.person.firstName); // Alice
console.log(data.person.age); // 30

// Convert to JSON String
console.log(JSON.stringify(data)); // {"person":{"firstName":"Alice","lastName":"Smith","age":30}}
```

```php
<?php
// JSON Data
$jsonData = '{"person": {"firstName": "Alice", "lastName": "Smith", "age": 30}}';

// Decode JSON
$data = json_decode($jsonData, true);

// Access Data
echo $data['person']['firstName']; // Alice
echo $data['person']['age']; // 30

// Encode to JSON
echo json_encode($data); // {"person":{"firstName":"Alice","lastName":"Smith","age":30}}
?>
```

FIN